

Memoria Técnica: API de Gestión de Usuarios y Control de Sesiones

1. Introducción

El objetivo principal de este proyecto es el desarrollo de una API REST segura y escalable que permita la gestión integral de usuarios y permisos. A diferencia de las implementaciones estándar de JWT, este proyecto busca resolver el problema de la **revocación de accesos en tiempo real**. La meta es proporcionar a los administradores una herramienta capaz de invalidar sesiones de forma inmediata ante cambios de roles, actualizaciones de seguridad o eliminación de cuentas.

2. Arquitectura de Datos

El sistema se fundamenta en un modelo relacional dentro de MongoDB utilizando tres colecciones clave:

- **User (Usuarios):** Almacena la información de perfil y las credenciales cifradas con `bcrypt`. Posee una relación de referencia hacia la colección de Roles.
 - **Role (Roles):** Define los niveles de autorización (`admin`, `user`). Esta separación permite una gestión de permisos granular (RBAC - Role Based Access Control).
 - **Token (Sesiones):** Es la pieza central de nuestra lógica de control. Registra cada JWT emitido, vinculándolo a un usuario y añadiendo un estado booleano `isValid`. Esta colección permite que el servidor tenga "memoria" de las sesiones emitidas.
-

3. Seguridad: Stateful JWT

La implementación de seguridad combina dos mundos:

1. **Stateless (JWT):** El cliente usa un token que contiene su identidad, evitando consultas constantes de perfil.
2. **Stateful (Validación en BD):** Hemos implementado un **Middleware de Doble Validación**.

¿Cómo funciona? Cuando llega una petición, el servidor realiza dos pasos:

- **Paso A:** Verifica que el JWT sea auténtico (firma y expiración).

- **Paso B:** Consulta la colección `Tokens` para verificar si ese token específico sigue marcado como `isValid: true`.

Si un administrador cambia el rol de un usuario o lo elimina, el sistema ejecuta un `updateMany` en la colección de tokens, marcando todas las sesiones de ese usuario como `false`. Esto resulta en una expulsión inmediata del sistema, superando la limitación tradicional de los JWT donde un token sigue siendo válido hasta que expira por tiempo.

4. Estructura del Proyecto

El proyecto sigue un patrón de diseño **MVC (Modelo-Vista-Controlador)** adaptado a una API:

Plaintext

```
src/
├── config.js      # Configuración de variables de entorno
├── database.js    # Conexión a MongoDB
├── app.js         # Configuración de Express
├── models/       # Definición de esquemas de Mongoose
│   ├── User.js
│   ├── Role.js
│   └── Token.js
├── controllers/  # Lógica de negocio (auth y usuarios)
│   ├── auth.controller.js
│   └── user.controller.js
├── services/     # Lógica auxiliar y persistencia
│   └── auth.service.js
├── middlewares/  # Validaciones de seguridad y JWT
│   └── authJwt.js
├── routes/       # Definición de rutas y endpoints
│   ├── auth.routes.js
│   └── user.routes.js
```

5. Guía de Uso con Docker

Para garantizar un entorno de ejecución idéntico y facilitar el despliegue, el proyecto está completamente dockerizado.

Requisitos previos:

- Docker y Docker Compose instalados.

Pasos para levantar el proyecto:

1. **Configuración:** Copia el archivo de ejemplo y rellena tus credenciales:

```
cp .env.example .env
```

2. **Despliegue:** Ejecuta el comando de construcción y levantamiento:

```
docker-compose up --build
```

3. **Acceso:**

API: <http://localhost:3000>

Mongo Express: <http://localhost:8081> (Para gestionar la base de datos visualmente).

6. Protocolo de Pruebas en Postman

Para validar el funcionamiento del sistema y la invalidación de sesiones, se han definido los siguientes flujos de trabajo:

A. Registro de Usuario (**Sign Up**)

Crea una nueva cuenta de usuario en el sistema.

- **Método:** POST
- **URL:** `{{url}}/api/auth/signup`
- **Cuerpo (JSON):**
- JSON

```
{
  "username": "tester",
  "email": "test@ejemplo.com",
  "password": "password123",
  "roles": ["user"]
}
```

B. Inicio de Sesión (**Sign In**)

Autentica al usuario y genera un token que se registra en la base de datos.

- **Método:** POST
- **URL:** `{{url}}/api/auth/signin`
- **Cuerpo (JSON):**
- JSON

```
{
```

```
"email": "test@ejemplo.com",
"password": "password123"
}
```

- **Nota:** Copia el **token** recibido para las siguientes peticiones.

C. Actualizar Usuario (**Update User**)

Permite al administrador cambiar roles o datos. **Esta acción invalida automáticamente todas las sesiones previas del usuario afectado.**

- **Método:** PUT
- **URL:** `{{url}}/api/users/<ID_DEL_USUARIO>`
- **Headers:** `x-access-token: {{token_admin}}`
- **Cuerpo (JSON):**
- JSON

```
{
  "roles": ["admin"]
}
```

D. Eliminar Usuario (**Delete User**)

Elimina al usuario de forma permanente e invalida cualquier token que pudiera tener activo.

- **Método:** DELETE
- **URL:** `{{url}}/api/users/<ID_DEL_USUARIO>`
- **Headers:** `x-access-token: {{token_admin}}`

8. Complicaciones y Desafíos Técnicos

Durante el desarrollo del proyecto, se presentaron varios retos técnicos que requirieron un análisis profundo y refactorización del código:

1. El Dilema de la Revocación del JWT (**Stateless vs. Stateful**)

- **El problema:** Por diseño, un JSON Web Token es válido hasta que expira. Si un administrador eliminaba a un usuario o le quitaba permisos, el usuario podía seguir operando hasta que el token caducara (ej. 24 horas después).
- **La solución:** Se optó por una arquitectura híbrida. Mantuvimos el JWT para el transporte de identidad, pero introdujimos la colección **Tokens** en la base de datos. El desafío fue no sobrecargar el sistema; se resolvió mediante un middleware ligero que solo consulta el estado de validez del token actual.

2. Violación del Principio de Responsabilidad Única (SRP)

- **El problema:** Inicialmente, la función `generateToken` se encargaba tanto de firmar el JWT como de guardarlo en la base de datos. Esto hacía que la función fuera difícil de testear y mantenía una dependencia oculta con la base de datos.
- **La solución:** Se refactorizó el código para separar las funciones. `generateToken` ahora solo firma el string, mientras que una nueva función `saveTokenSession` se encarga de la persistencia. Esto hace que el servicio sea más modular y fácil de mantener.

3. Sincronización del Middleware `verifyToken`

- **El problema:** Al introducir la validación en la base de datos, existía el riesgo de bloquear a usuarios legítimos si la sesión no se encontraba por errores de red o registros antiguos.
- **La solución:** Se implementó una lógica de "aduana" en el middleware `authJwt.js`. Ahora, el sistema verifica primero si la sesión existe y está activa antes de proceder a la decodificación del JWT. Si el Admin actualiza al usuario, el middleware captura el cambio de estado inmediatamente.

4. Gestión de Sesiones "Huérfanas"

- **El problema:** Al eliminar un usuario, sus registros en la colección `Tokens` quedaban ocupando espacio innecesario y técnicamente válidos (aunque el usuario ya no existiera).
- **La solución:** Se ajustó el controlador `deleteUserById` para que, además de borrar el perfil, ejecute una limpieza (o invalidación) de todos los tokens asociados a ese `userId`.